General Notes

- always check for `null` objects (`compareTo` etc)
- `compareTo()` API. Negative → "this" is less. Positive → "this" is greater
- `Integer.compare(int x, int y)` Negative → `x` is less
- `inoremap jj <Esc>`
- `java Main < inputs/tc > OUT`
- `vim -d file1 file2`
- Immutable → you can repeat operations with no cost
- Generics → remember functions can call itself
- Stream → you can use old functions

Notes on abstract

- can have both abstract and concrete methods
- can have constructors
- NO private abstract, NO instantiation

Syntax

```
// comment ot justify
@SuppressWarnings("unchecked")
Queue<Pasenger>[] temp = (Queue<Passenger>[]) new Queue<?>[n];
this.passengers = temp;
```

```
class MyException extends Exception() {
    public MyException(String msg) {
        super(msg);
    }
}
```

Code	Description
`%s`	String
%d	Decimal (integer)
%f	floating-point number
%.3f	float to 3 dp
%b	boolean
%n	newline
\	escape character works for `" ' \ n t b r f`
%c	character

Stream

```
// find all pairs of stations that share common busn mae
// using indexOf, chain flatmaps, anyMatch
return stations.stream()
    .flatMap(s1 -> stations.stream()
        .filter(s2 -> stations.indexOf(s1) <
stations.indexOf(s2))
        .filter(s2 -> buses.stream()
            .anyMatch(bus -> s1.buses().contains(bus) &&
s2.buses().contains(bus))
        .map(s2 -> List.of(s1, s2)))
    .toList();
// reduce anyMatch to a boolean so its like
// filter(x -> true)
// altneratively instead of anymatch, u can try filter +
limit(1)
```

ShipBundle example

```
// input: id_one, id_two, qty
// target: ship if both allowed
// strategy: 1. double for loop 2. convert into pair 3. check
```

```
pass, FLATMAP
Maybe.of(this.products.get(id_one))
    .filter(one -> one.canShip(qty))
    .flatMap(one -> Maybe.of(products.get(id_two))
        .filter(two -> two.canShip(qty)) // two also passed!
        .map(two -> new Pair<>(one, two)))
    .ifPresent(pair -> applyFunction(pair));
// one -> Maybe.Of()...
// is a transformer returning Maybe.of()
// if we return Maybe.of() inside map, we want to use FLATMAP
```